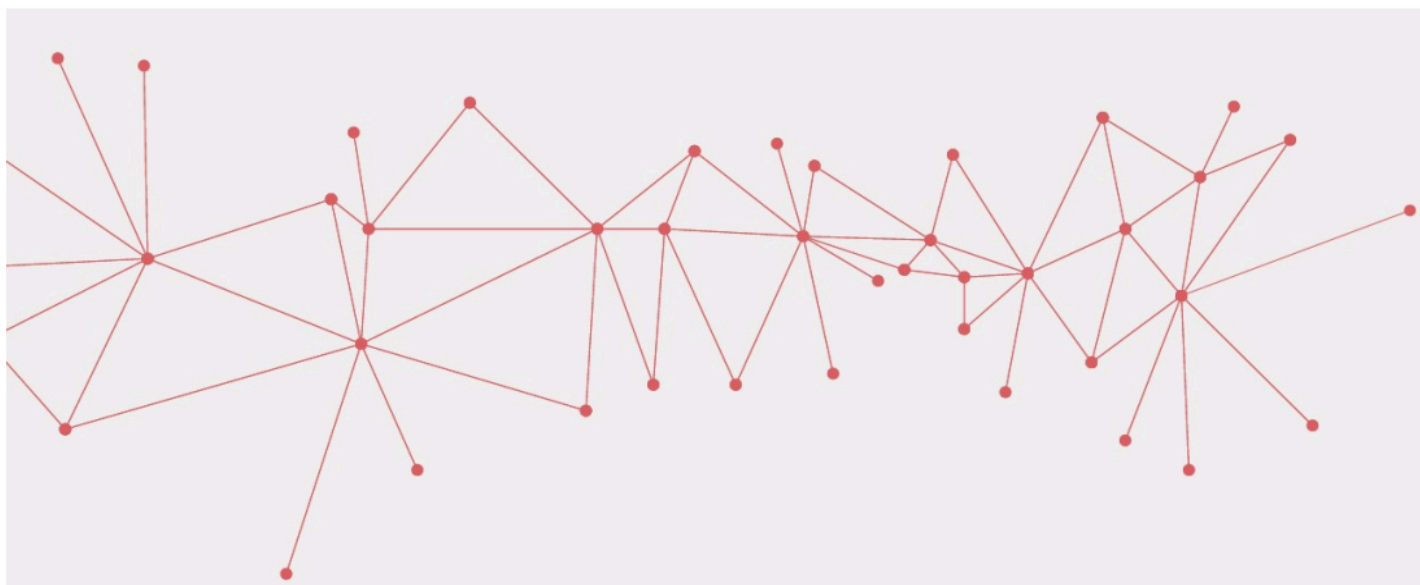




Λ M V Λ R Λ

PERFORMANCE TEST

GEMUE – Demo Environment November 2024



Objective

"You can't test everything, but you can test the parts that matter." - various authors.

To evaluate the performance and stability of the GEMUE IBM Cognos Systems under varying user loads requesting only data for a dashboard.

The tests simulate different levels of concurrent user activity to identify the system's breaking points and performance characteristics.

Not in scope is the load generated by “standard” reports.

This documentation and related work was done by: Sascha Papevis and Ralf Roeber.

Table of Contents

Objective..... 2

What is a concurrent user?..... 4

A word on IBM Cognos..... 5

How to Execute the Load Test with Different Parameters..... 6

How to choose the correct parameters..... 7

Why repeat Tests?..... 8

Running Tests..... 9

Analyzing the results.....10

Analysis Approach..... 14

How to edit the test plan..... 15

What is a concurrent user?

Definition: Concurrent users (CU) refer to the number of users who are actively using a system or application at the same time.

In terms of jMeter CU equals to threads.

This metric is important for understanding the real-time load on a system and for planning capacity and scalability.

In a system that is used on a daily basis, like a Sales and Order system, the load is distributed evenly. Anyhow, it has to withstand peaks. A monthly reporting system will be heavily used at the end and beginning of each month, but see almost no load the rest of the time.

The system should be tested having the peak usage in mind, not the average.

To calculate the concurrent users we need two things:

1. The total number of visits for a short time period when your site/app is at peak traffic levels. This can be found via e.g. Google Analytics, AppDynamics by seeing what the highest number of visits was for a single hour in the course of e.g. a month. Look at the day that has seen the highest number of visits, and drill down to see what hour of that day was the busiest and how many visits you had during that hour. Note this value down. I will call this value "peak_hourly_visits" in this text.
2. The average time a user spends interacting with your site/app. In Google Analytics this is called "Average session duration" and we will call it that in this text also, but sometimes it is called "Average time on site". If this value changes a lot for your site/app depending on which time period you look at you might want to use one of the larger values you find, to be on the safe side. We want all times in seconds, so if e.g. Google Analytics tells you "00:03:19" (3 minutes, 19 seconds) you should note down 199 as the average session duration.

Those two values are used in the following formula to calculate the number of concurrent users to use in your load test:

$$\text{concurrent_users} = (\text{peak_hourly_visits} * \text{average_session_duration}) / 3600$$

Provided that each simulated user (VU) in our load test behaves realistically (i.e. simulates a real user well), we will now be able to stress the site/app with a similar kind of traffic that it normally only sees during peak traffic hours.

[Source for above](#)

Alternatively use rule of thumb (Ralf): 3% to 10% of the user base.

A word on IBM Cognos

IBM Cognos has high and low affinity processes. These directly define the limit of the system and protect it from overloading while limiting the usage.

For the reporting service:

Number of high affinity connections for the report service during non-peak period	2
Number of low affinity connections for the report service during non-peak period	8
Maximum number of processes for the report service during non-peak period	2

Depending on this setting the report services will behave.

For dashboards there are no limits that are easily accessible. Anyhow the setting of java memory, threads and more have a influence on the overall performance and are directly impacted by the dashboard usage.

Therefore it is a very good idea to focus on load testing the dashboard performance as it directly impacts the performance for the overall system.

How to Execute the Load Test with Different Parameters

In general we want to achieve a load test duration of around 30 minutes per test execution.

Each load test has three parameters:

1. Number of parallel users (running threads)
2. Number of times each user will execute the same test (loops)
3. Time period to ramp up the users (ramp up time)

The script to execute the tests uses the parameters as follows:

```
& .\Run.ps1 -p="$PASSWORD" -t <threads> -l <loops> -rup <ramp up time>
```

The following parameters will execute 10x users (threads) with 16 loops and 480 seconds (8 minutes) ramp up time.

```
& .\Run.ps1 -p="$PASSWORD" -t 10 -l 16 -rup 480
```

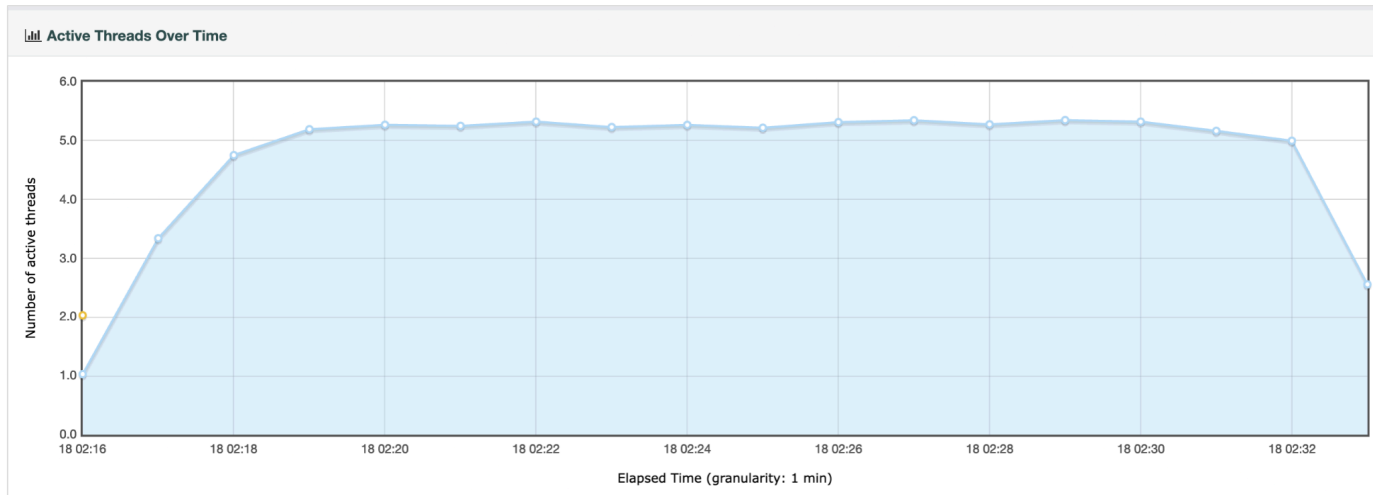
How to choose the correct parameters

Depending on the duration of each test and the number of loops together with the ramp up time, users (threads) may finish testing already before all users have been ramped up.

Looking at the executions done on 18th of November, we used 50 threads, 3 loops with 960 seconds ramp up.

We can see that 2 threads finished already, after starting 6 threads at the beginning of the logfile.

This results in around five active threads over time.



Above screenshot shows a nice ramp up and ramp down phase including a period of 15 minutes of continuous load with 5 users (threads).

This is a good result.

Why repeat Tests?

In order to avoid wrong test results because of network problems or issues with the test client itself, we recommend repeating each test cycle at least 3 times.

```

echo "AMVARA Load Test - Threads, 10, Loops 16, RU 480"
echo "=====
echo "=====
echo "=====
& .\Run.ps1 -p="$PASSWORD" -t 10 -l 16 -rup 480
& .\Run.ps1 -p="$PASSWORD" -t 10 -l 16 -rup 480
& .\Run.ps1 -p="$PASSWORD" -t 10 -l 16 -rup 480
sleep 600
  
```

The above test execution provides 3x test results. While analyzing the result, we check for irregularities and may discard results as invalid or not useful.

In order to distinguish test cycles, we use sleep 600 seconds (=10 minutes) in between. This also provides the system under test with the time to free up buffers and memory.

A normal load test execution runs for 12 or more hours.

Running Tests

Useful to smoke test and confirm if the application and Test Plan are currently compatible.

1. Open the script in Notepad++ as an Administrator
E:\LoadTest\test_plan_run.ps1
2. Validate the Password
3. Before running the script, edit it to uncomment the desired configuration:
 - **One User**
 - **Small (10 Users Once)**
 - **Small (15 Users Once)**
4. Open PowerShell as an Administrator
5. Navigate to the directory
E:\LoadTest\
6. Run the script
E:\LoadTest\test_plan_run.ps1

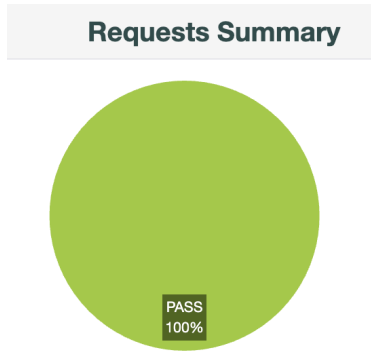
Analyzing the results

- JMeter dashboards are generated in the directory:
E:\LoadTest\logs_dashboard.
- Each test produces a **browser-viewable dashboard directory** and an **archive file** for download and review

1. Navigate to the dashboard directory
E:\LoadTest\logs_dashboard
2. Navigate the directory of the dashboard you intend to review
3. Open the file index.html in a browser.

Request Summary

Shows if there have been failures. As soon as there are failures, we can summarize that there have been issues.



Dashboard Page

- Locate the last table, which presents requests grouped by labels.
- Sort the table in descending order by **average request duration** to identify performance bottlenecks.

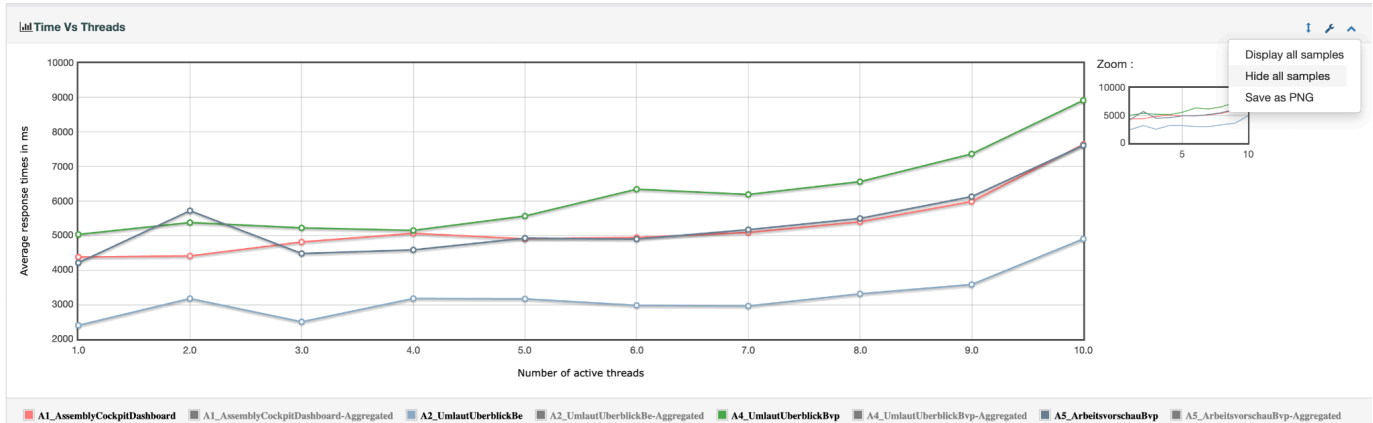
Statistics										
Requests		Executions			Response Times (ms)					
Label		#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct
Total		84484	0	0.00%	69.37	0	5201	49.00	82.00	89.00
A6_UmlautUberblickOft		160	0	0.00%	8894.49	5956	14321	8585.00	11660.40	13530.40
A4_UmlautUberblickBvp		160	0	0.00%	6710.06	3854	12016	6353.00	9103.90	10007.30
A7_ArbeitsvorschauOft		160	0	0.00%	6165.13	4391	11298	5865.00	7804.70	9036.50
A1_AssemblyCockpitDashboard		160	0	0.00%	5684.83	3976	10951	5334.50	7409.30	9152.40
A5_ArbeitsvorschauBvp		160	0	0.00%	5666.13	4153	10949	5271.00	7903.60	9791.20

In this case we see for A6_UmlautUberblickOft

- Average: 8.8 seconds

- Max: 14.3 seconds

This indicates that the system response times are slower when more users are on the system. Which indicates a performance issue.



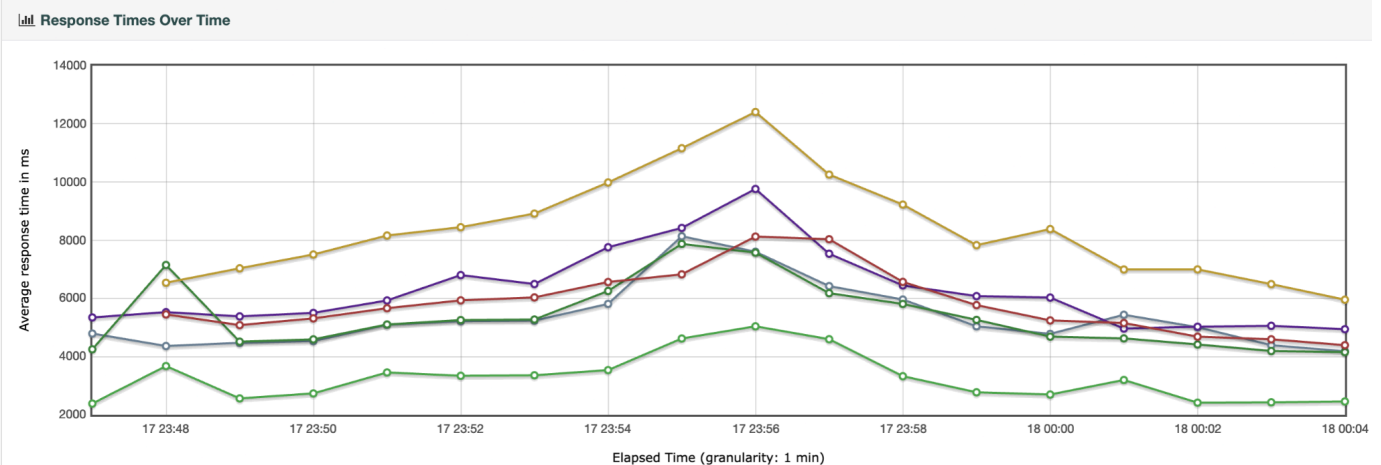
Looking at Response Time vs Threads, we can see that 8 parallel users already have an impact on the system.

Charts -> Over Time Page:

1. Response Times Over Time

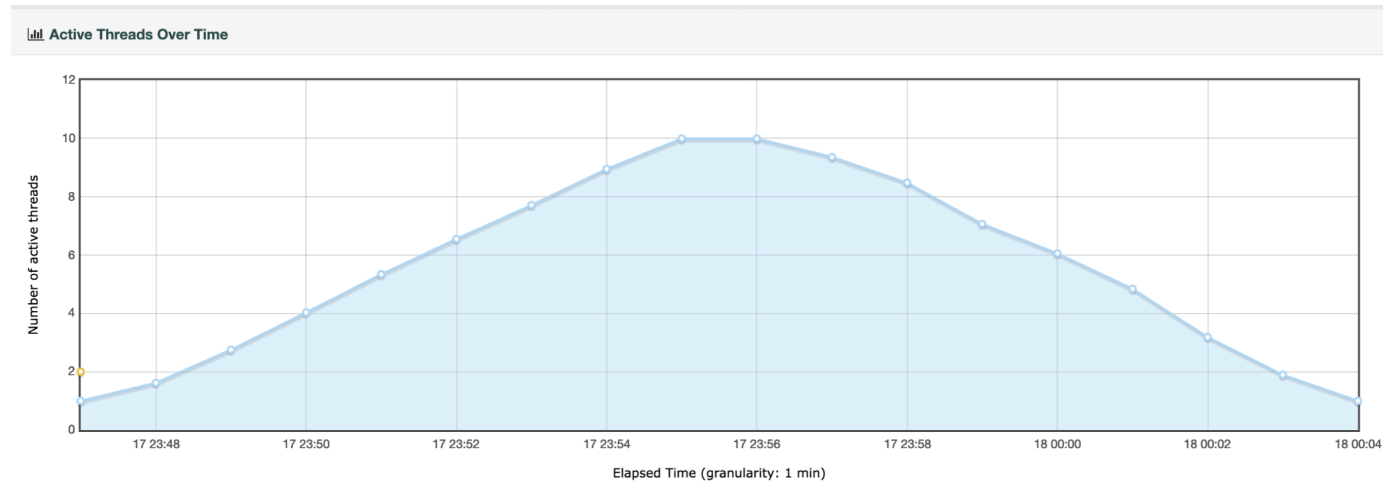
Use the wrench button to:

- **Hide all samples.**
- **Enable samples prefixed with "A":**
 - **A-level samples:** Represent top-level request groups.
 - **B-level samples:** Represent middle-level request groups.
 - **C-level samples:** Represent individual atomic requests.
- Enable samples sequentially (A → B → C) to observe how higher-level request groups affect the performance of individual requests.



The chart shows that response times have a hiccup at the beginning - may be for extra caching. Repeating the test would show if the same problems persists. In the middle part we see that response times grow, which indicates a performance degradation. We should not put so many users on the system. Ideally the lines are flat over time.

Threads over time



Active threads over time should show a smooth ramp up and ramp down curve.

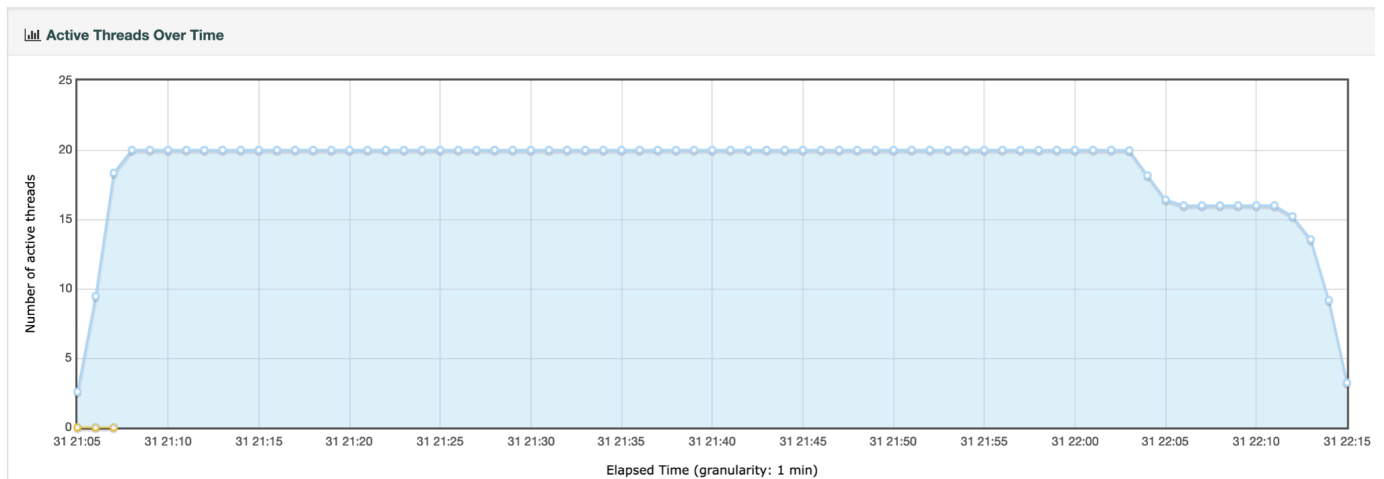
The screenshot above shows a test result with issues in the parameters:

- overall test period is too small - it should be around 30 minutes
- the time frame with max users is only 1 minute - should be 15 to 20 minutes
- the ramp up time could be shorter

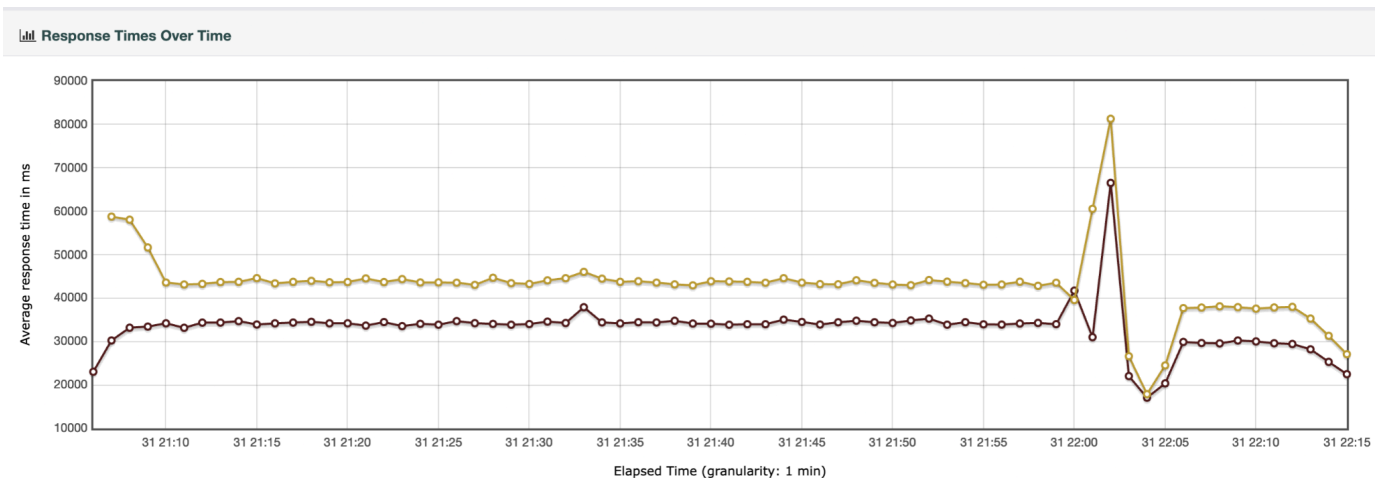
Test was executed with: 10 threads, 16 loops and 480 ramp up time

Better use: 64 loops or 128 loops to give the users a longer time period in the system.

Another example:



This screenshot shows a nice ramp up and constant number of threads in the system until the end of the test.



The response time over time shows a hiccup in the system towards the end of the test. This could be caused by an issue on the database, the network, the client or the application server and needs to be investigated together with the system logs of CPU, Disk I/O and Network I/O.

We recommend having a system monitoring in place that can provide insights to CPU, RAM, Network and Disc IO.

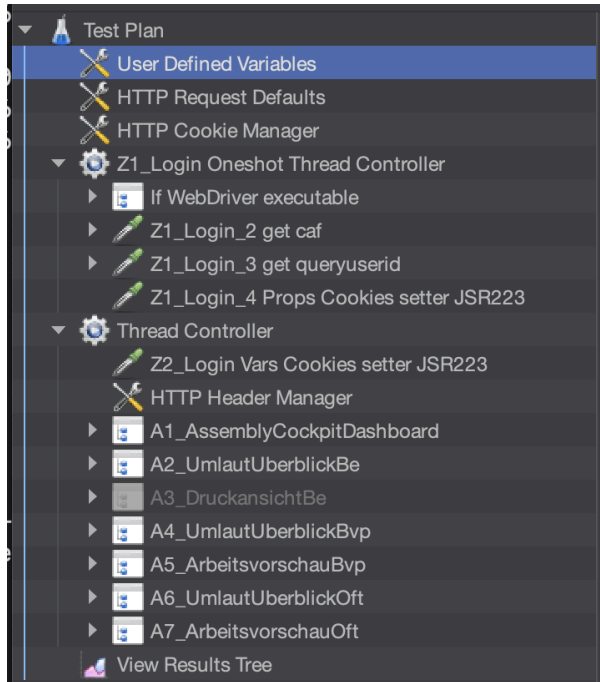
Analysis Approach

- Create a baseline, which shows good performance on all samples
- Run different test scenarios
- Focus on **request durations average versus maximum** to identify problem areas.
- Compare insights across **hierarchical levels** (A, B, and C) to determine which group or request significantly impacts overall performance
- Check that threads over time show a “mountain”
- Test often, take your time for analysis

How to edit the test plan

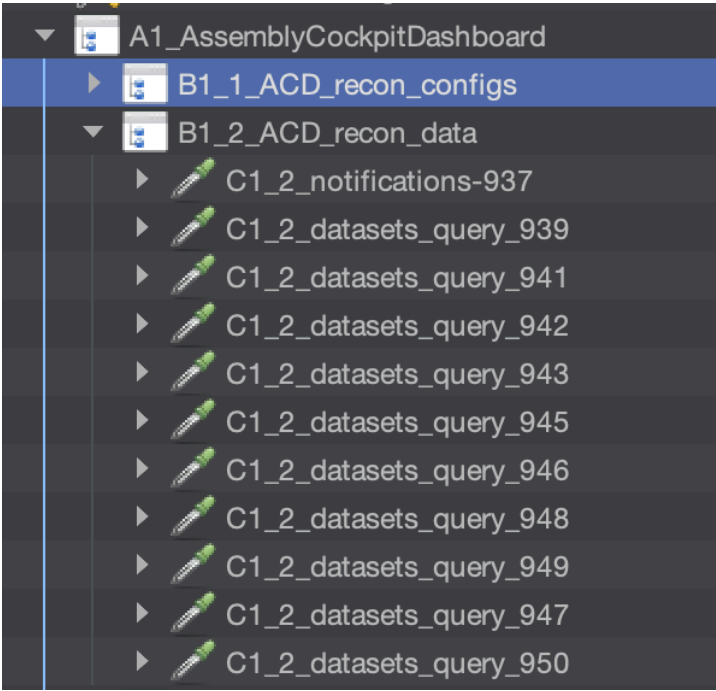


The archive GEMUE.jmx contains the test steps executed.



Z1_Login is executed only one time and prepares the security tokens while logging into the system.

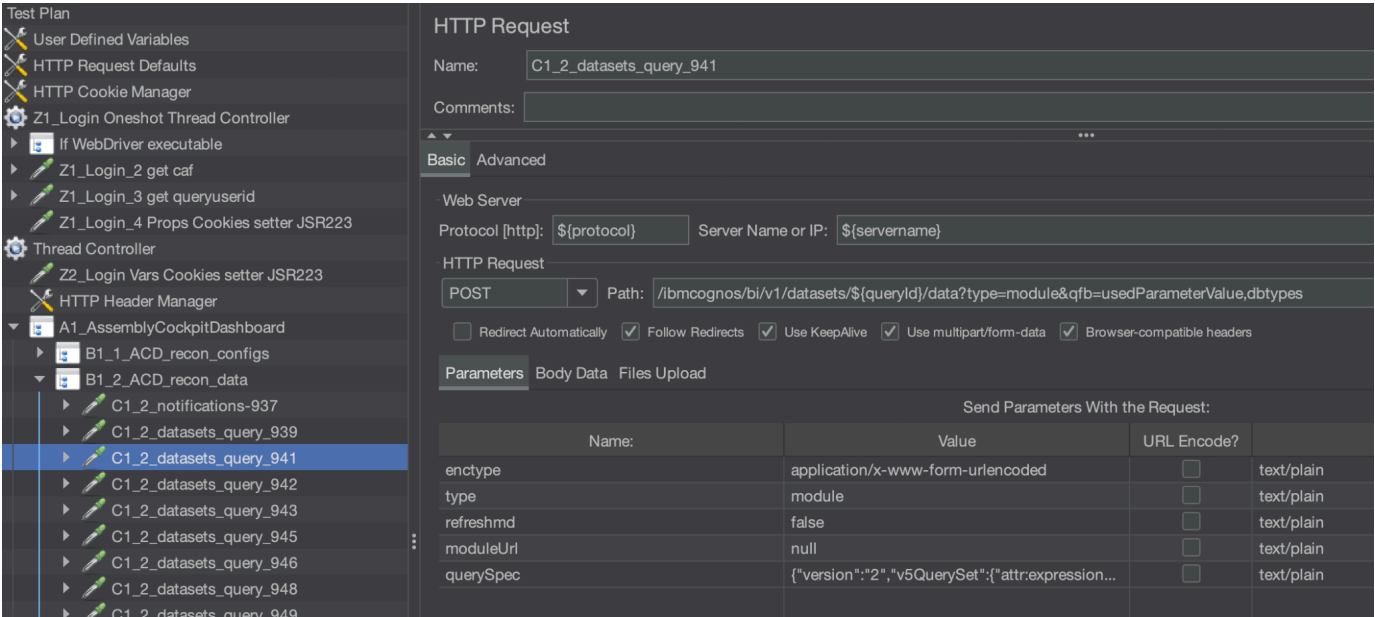
The thread Controller contains the actual requests to put load onto the system under test.



Test steps are grouped using transaction controllers. All requests represent requests against dataset queries.

Javascripts, CSS files ... are not part of the load test.

Each controller executes different requests against the Cognos backend to retrieve data





AMVARA

<https://www.amvara.de/>

info@amvara.de

+34 717 102 603